A Random Forest for Network Intrusion Classification on UNSW-NB15

26989395
RW741

***Abstract*—The UNSW-NB15 dataset holds 257,673 network flow records in ten classes of very different sizes. Almost half of the records are duplicates, and about one in six sits in a group of identical records with conflicting classes. A random forest was implemented and tuned on five-fold cross-validation through five research questions set before the experiments. It was compared with the classification tree and the k-nearest neighbours classifier of assignment 2. The selected forest scores 0.6102 on the macro-averaged F_1 score, against 0.5860 for the tree and 0.5010 for the k-nearest neighbours classifier, and scores higher than both on all five folds. The best forest draws more than twice the usual number of features at each split and uses no bootstrap. None of the four imbalance treatments raised the score. Backdoor, Analysis and DoS, whose records mostly sit in conflicting groups, remain the weakest classes, because no classifier can separate records with identical features.**

***Index Terms*—Random forest, ensemble learning, intrusion detection, class imbalance, UNSW-NB15**

I. Introduction

An intrusion detection system decides which of the exchanges crossing a network are attacks. In assignment 2 this decision was treated as a classification problem on the UNSW-NB15 dataset [18]. Two models were compared there. The classification tree scored 0.5872 on the macro-averaged F_1 score against 0.5010 for the k-nearest neighbours classifier, but it also showed the signs of high variance. An unconstrained tree scored 0.7806 on its training records and 0.5791 on the validation records, and three features carried roughly two thirds of its decisions.

A random forest is designed to reduce the variance of a single tree [4]. It grows many trees, each on a random sample of the records and each choosing its splits from a random subset of the features, and classifies a record by the majority vote of the trees. Averaging over trees that make different errors cancels part of the variance of a single tree, and restricting the features at each split stops every tree from being built around the same three features. The random forest was chosen over other ensemble methods because it is an ensemble of the same classification tree. The comparison with assignment 2 therefore changes one thing, the combining of trees, where boosting or a stacked model would change the underlying model as well.

The tree, the forest and the resampling methods were implemented for this report. The forest was tuned and evaluated on the same five cross-validation folds as assignment 2, so the forest is compared fold by fold with both earlier models. The experiments answer five research questions, set out in Section IV-A. They concern the number of trees, the two sources of randomness, the depth and leaf size of the trees, the treatment of the class imbalance and the choice of performance measure.

The forest scores 0.6102 on the macro-averaged F_1 score, against 0.5860 for the tree, retuned for this report, and 0.5010 for the k-nearest neighbours classifier, and scores higher than both on every fold. With the usual six features per split the forest scored below the single tree, and the random sample of records lowered the score at every number of features below 42. The classes whose records share their features with records of other classes remain the weakest for every model, as they were in assignment 2.

Section II gives the background on the dataset, the models, the treatment of imbalance and the measures. Section III describes the implementation and its verification, Section IV the research questions and the protocol, and Section V the results and their discussion. Section VI concludes.

II. Background

This section describes the dataset and the two models of assignment 2, then defines the ensemble methods, the treatments of class imbalance, the performance measures and the statistical procedures used in the later sections.

A. Network intrusion detection and the UNSW-NB15 dataset

A network flow record summarises one exchange between two hosts. The record holds counts and timings, such as the bytes sent each way and the duration, rather than the contents of the exchange. An intrusion detection system reads such records and decides which of them represent an attack. Treating this decision as a classification problem is the usual approach [6]. Each record is an instance, the counts and timings are the descriptive features, and the class is the kind of attack the record represents, or ordinary traffic if it represents none.

The UNSW-NB15 dataset was created at the Cyber Range Lab of UNSW Canberra [18]. The IXIA PerfectStorm tool generated a mix of real ordinary traffic and synthetic attacks, and the captured traffic was summarised into flow records. The 257,673 records used here are the published training and testing sets combined. Each record has 42 descriptive features and one of ten classes. Three of the features are nominal: the protocol, the service and the connection state of a flow. The classes are Normal and nine attack categories: Generic, Exploits, Fuzzers, DoS, Reconnaissance, Analysis, Backdoor,

Shellcode and Worms. The class sizes range from 93,000 Normal records to 174 Worms records, a ratio of about 534 to one.

B. The k-nearest neighbours classifier

The k-nearest neighbours classifier stores the training records and does nothing else until it is asked to classify [9]. To classify a record, it measures the distance from that record to every stored record, takes the k closest, and assigns the class that most of them hold.

Two choices define the classifier. The first is k. A small k follows the fine detail of the boundary between classes but lets a single misleading record decide, while a large k steadies the answer and blurs any narrow boundary. The second is the distance itself. The Manhattan distance, the sum of the absolute differences, is often preferred to the Euclidean distance when there are many features [1].

The distance sums a contribution from every feature, so a feature measured in large units contributes more than a feature measured in small ones, whatever either feature means. Features must therefore be brought onto a comparable scale before the distance is meaningful. Fitting costs nothing, and every prediction costs a pass over the stored records.

C. Classification trees

A classification tree splits the training records into two groups by testing one feature, then splits each group again, and continues until a stopping rule applies [5]. Each leaf holds the records that reached it, and a new record is classified by following the tests down to a leaf and taking the majority class there.

The test at each node is chosen to make the resulting groups as pure as possible, where a pure group holds records of one class only. For a group S in which a proportion p_c of the records belong to class c, the Gini index and the entropy are

$$G(S) = 1 - \sum_c p_c^2, \qquad H(S) = -\sum_c p_c \log_2 p_c. \tag{1}$$

Both are zero for a pure group and largest when every class is equally common. The Gini index comes from [5] and the entropy from [21], and the two rarely choose different splits [22]. A split of S into S_L and S_R is scored by the decrease in impurity it achieves, called the information gain when the impurity is the entropy:

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R), \tag{2}$$

where I is either measure and $|S|$ is the number of records in S. The tree takes the split with the largest decrease at each node and never revisits it, so the search is greedy.

On its own, a tree keeps splitting until every leaf is pure, which usually means it has memorised the training records rather than learned anything general. Growth is therefore constrained: a maximum depth limits the number of tests on the path from the root to a leaf, and a minimum leaf size stops a split that would leave too few records behind.

Even when constrained, a tree has high variance. A small change in the training records can change the split chosen near the root, and every split below it changes with it. Ensembles of trees address this weakness.

A split depends only on the order of the values of a feature, so rescaling a feature has no effect on the tree, unlike the k-nearest neighbours classifier.

The importance of a feature is the total decrease in impurity of the splits that test it, weighted by the number of records each split divides, and divided by the total over all features so that the importances sum to one. This measure favours features with many possible split points, because a feature with more distinct values offers more chances to find a good split by luck [23].

D. Ensemble learning

An ensemble combines the predictions of several models into one [11]. For classification, the members are usually combined by a majority vote, which helps only when they make different errors. The members must therefore be diverse, meaning that their errors are at least partly independent.

A deep tree has low bias and high variance: it can follow almost any boundary, but follows a different one for every sample. Averaging many deep trees lowers the variance and leaves the bias about the same [13].

The gain from averaging is limited by how alike the members are. Consider B models whose predictions each have variance σ^2, and let ρ be the correlation between the predictions of any two of them. The variance of their average is [13, Eq. 15.1]

$$\rho\sigma^2 + \frac{1-\rho}{B}\,\sigma^2. \tag{3}$$

The second term shrinks towards zero as B grows, but the first does not depend on B. Adding members therefore helps only up to a point, after which the only way to lower the variance further is to lower ρ, the correlation between the members.

Ensemble methods differ in how they make their members diverse. Bagging trains each member on a different random sample of the records and gives every vote the same weight. Boosting trains the members one after another, each giving more weight to the records that the earlier members misclassified. Stacking trains a second model to combine the predictions of several different kinds of model. Bagging lowers the variance of its members, while boosting mainly lowers their bias. This report therefore takes the bagging family, because the weakness of the assignment 2 tree is its variance. Everything that the random forest adds to bagging is there to lower ρ.

E. Bagging and random forests

Bagging, short for bootstrap aggregating, is the simplest method of the family [3]. Each tree is grown on a bootstrap sample, which is a sample of n records drawn with replacement from the n training records. A record is missed by one draw with probability $1 - 1/n$, and by the whole sample with probability $(1-1/n)^n$, which is about $1/e \approx 0.368$ for large n.

Each tree therefore sees about 63.2% of the distinct records, and about 26.4% of the records appear two or more times. The trees are then combined by a majority vote.

Bagging changes only the records that a tree sees. When a few features separate the classes far better than the rest, every tree still splits on those features near its root, so the trees stay alike and their correlation ρ stays high. The random subspace method instead grows each tree on a random subset of the features [15]. The random forest combines the two ideas [4]. Each tree is grown on a bootstrap sample, and at every node the split is chosen from a new random subset of m features rather than from all of them. The usual value of m for classification is the square root of the number of features, which is about six of the 42 features here.

The number of features per split sets a trade-off. A small m makes the trees more different from one another, which lowers ρ. Each tree is weaker, however, because the best feature for a split is often not among those drawn, which raises the variance σ^2 of each tree and the bias. A large m gives strong but similar trees, and the best value lies between the two and depends on the data. With all 42 features at every split the forest becomes bagging. With all 42 features and no bootstrap, every tree is grown on the same records with the same choices, so every tree is the same tree.

The bootstrap is not the only source of diversity. Extremely randomised trees drop the bootstrap and grow every tree on all of the records, relying instead on random features and random thresholds at each split [12]. The forest selected in this report also has no bootstrap but keeps the best threshold at each split, so its only source of diversity is the random set of 14 features drawn at each split.

Adding trees does not make a random forest overfit [4]. The score rises quickly over the first trees and then levels off [20], so the number of trees trades computing time against a shrinking gain.

F. Learning from imbalanced classes

A classifier that minimises the number of errors favours the common classes, because an error on a rare class costs no more than an error on a common one and the common classes supply most of the records [14]. Treatments of imbalance either change how much each record counts, or change the records themselves.

Class weighting gives each record a weight equal to the inverse frequency of its class, so that a rare class counts for as much in the impurity as a common one. The balanced random forest changes the sample of each tree instead [8]. Each tree draws the same number of records from every class with replacement, usually as many as the smallest class holds, so that every tree is grown on balanced data.

Resampling methods change the training records before any model is grown. SMOTE adds synthetic records to a rare class [7]. For each new record, it picks a record of the class and one of its k nearest neighbours from the same class, and places the new record at a random point on the line between the two. A nominal feature of the new record takes the level held by most of the k neighbours. The rare class is therefore enlarged with records that resemble its own, rather than with exact copies, which lead a tree to grow small regions around the copied records.

A Tomek link is a pair of records from different classes in which each record is the nearest neighbour of the other [24]. Either one record of the pair is noise, or both lie on the boundary between the two classes, and removing both records therefore cleans the boundary. SMOTE followed by the removal of Tomek links combines the two methods, so that a synthetic record placed among the records of another class is removed together with its neighbour from that class [2].

Records with identical features but different classes are at distance zero from each other, so they always count as Tomek links. Removing the links therefore removes every such conflict from the training records.

G. Performance measures

Accuracy is the proportion of records classified correctly. When one class holds most of the records, accuracy mostly reports how well the majority class is handled, and a model that ignores every rare class can still score well [14]. In this dataset Normal and Generic together hold 58.94% of the records, so accuracy mostly reflects these two classes.

Precision and recall describe a single class. For a class c, let TP_c be the number of records of class c assigned to it, FP_c the number of records of other classes assigned to it, and FN_c the number of records of class c assigned elsewhere. Then

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F_{1,c} = \frac{2P_cR_c}{P_c + R_c}. \tag{4}$$

Precision is the share of the records assigned to the class that belong to it, and recall is the share of the records belonging to the class that were assigned to it. The F_1 score is their harmonic mean, which is low unless both are reasonable.

Per-class scores are combined in one of two ways. A weighted average weights each class by its size, and therefore reports much of what accuracy reports. A macro average takes the plain mean over the C classes,

$$F_1^{\text{macro}} = \frac{1}{C}\sum_c F_{1,c}, \tag{5}$$

so every class counts alike regardless of size. Balanced accuracy is the macro average of recall alone. The macro-averaged F_1 score is the primary measure in this report, because a model that misses the rare attack classes should score poorly. Accuracy, the weighted F_1 score and balanced accuracy are reported as secondary measures.

H. Model selection and statistical comparison

In k-fold cross-validation the records are divided into k parts, and each part is used for evaluation once while the other $k-1$ parts train the model [17]. The result is k scores whose spread indicates how stable the estimate is. Stratified folds

hold the proportion of each class fixed in every part, so that every part holds a similar share of even the smallest class.

Cross-validation is also used to choose between configurations of a model. The fold scores carry noise, so the highest mean score is partly a matter of chance. The one-standard-error rule therefore finds the best configuration, then selects the simplest configuration whose mean score lies within one standard error of the best [13, Section 7.10]. The standard error is the standard deviation of the fold scores divided by the square root of the number of folds. A configuration within one standard error of the best cannot be told apart from it on these folds, and the simpler one costs less to fit and to use.

When two models are evaluated on identical folds, their scores are paired: the two scores for a fold differ only by the model, so a paired test can compare them while ignoring how much the folds differ from one another [10]. The paired t-test takes the difference d_i between the two scores on each fold i, and tests whether the mean difference $\bar{d}$ is zero using

$$t = \frac{\bar{d}}{s_d / \sqrt{k}}, \tag{6}$$

where s_d is the standard deviation of the k differences, with $k-1$ degrees of freedom. The test assumes that the differences are independent. They are not, because in five-fold cross-validation any two training sets share three quarters of their records. The test therefore underestimates the variance of the difference, and its p-values are too small [10], [19].

Running two tests at the 5% level gives about a 10% chance that at least one of them reports a difference that does not exist. The Holm correction controls this chance [16]. With m tests, it multiplies the i-th smallest p-value by $m - i + 1$ and raises any adjusted value that falls below the one before it, so with two tests the smaller p-value is doubled. It is less strict than the Bonferroni correction, which multiplies every p-value by m, and still holds the chance of any false difference at 5%.

III. Implementation

The classification tree, the random forest, SMOTE and the removal of Tomek links were implemented in Python with NumPy, with the inner loops compiled by Numba. The k-nearest neighbours classifier is the model of assignment 2, reused unchanged. The code runs on Linux and is available in a public repository.[1]

A. Data preparation

The data were cleaned as in assignment 2. The absent values of the service feature were replaced by the level `none`, the binary flag `is_ftp_login` was reduced to 0 and 1 by mapping every value above zero to one, and the record identifier was removed. The cleaned dataset holds 257,673 records described by 42 features: 37 numeric features, the three nominal features and two binary flags.

Many records are not unique. Ignoring the class, 15,712 combinations of descriptive features occur more than once, and together they cover 119,701 records, or 46.5% of the dataset. Of these combinations, 2,445 carry conflicting classes, meaning that records with identical descriptive features are labelled as different classes. Those groups hold 40,400 records, or 15.7% of the dataset. Nothing in the dataset resolves the conflicts, and they cap the accuracy any classifier can reach at 0.9139. The conflicts fall unevenly across the classes, as shown in Table IX. Backdoor, DoS and Analysis have 84.9%, 77.4% and 77.0% of their records in conflicting groups, while Normal, Shellcode and Generic have 1% or less.

The tree receives every feature as integer codes, fitted on the training fold. A numeric feature receives one code per distinct value, in the order of the values, and a nominal feature receives one code per level. No logarithm or scaling is applied, because a tree depends only on the order of the values, as described in Section II-C. Grouping each numeric feature into 255 bins of equal record count was tried first, since it makes the split search faster. It lowered the macro-averaged F_1 score of the tree to 0.5717, against 0.5872 for the assignment 2 tree, so the split search is exact.

The k-nearest neighbours classifier keeps its assignment 2 inputs. Ten correlated features were removed, the numeric features were transformed by $\log(1+x)$ and standardised, and the nominal features were one-hot encoded, with levels held by fewer than 1% of the records collapsed into one level. Every step was fitted on the training fold only.

B. Classification tree

The tree is grown depth first, with an explicit stack of the nodes still to be split in place of recursion. A node becomes a leaf when it holds records of one class only, when it is at the maximum depth, or when it holds fewer than twice the minimum leaf size, since no split could then leave enough records on both sides. A leaf stores only its majority class, which keeps a forest of 500 trees small enough to hold in memory.

Every record carries a weight, which is the number of times the record was drawn into the sample of the tree multiplied by the weight of its class. A bootstrap sample or a class weighting therefore changes only the weights, and no record is copied. A record that was not drawn has weight zero and is left out of the tree.

At each node, the split search builds a class histogram for each candidate feature, holding the weighted count of each class at every code present at the node. For a numeric feature, the histogram is swept from the lowest code to the highest, and every boundary between two occupied codes is a candidate threshold. The threshold stored is halfway between the two codes. For a nominal feature, every level present is tried against the rest, and a level not seen in training follows the rest at prediction time. A split is made only if it lowers the weighted impurity of the node, and the decrease is added to the importance of the feature it tests.

When the number of features per split is limited, the features are visited in a random order at each node, and the search stops after that number of features. If none of them gives a

[1] https://github.com/andrewcottrell/machine_learning_assignment_4

split that lowers the impurity, the search continues through the remaining features, so a node becomes a leaf only when no feature can split it.

C. Random forest

The forest grows each tree with the tree of Section III-B, changing only the record weights and the number of features per split. Three bootstrap options set the weights. The standard bootstrap draws n records with replacement from the n training records, and the balanced bootstrap draws the same number of records with replacement from every class. With no bootstrap, every record has weight one. Class weights are computed once from the training fold, as the inverse frequency of each class, and are the same for every tree.

The forest classifies a record by the majority vote of its trees, and a tied vote goes to the class with the lowest code.

The trees are built in parallel, each with its own seeds derived from the forest seed and the index of the tree, so a forest does not depend on the number of cores that built it.

The votes are added one tree at a time, so the predictions of the first one, two, five and more trees are available from one fitted forest. The curve of the score against the number of trees therefore costs a single fit of the largest forest. The importance of a feature in the forest is the mean, over the trees, of its importance in each tree, where the importances of each tree are normalised to sum to one.

D. Resampling

SMOTE was implemented for records with both numeric and nominal features. Distances are measured on the numeric features after the logarithm and the standardisation of the k-nearest neighbours pipeline, fitted on the training fold. A nominal feature or binary flag on which two records differ adds one to the squared distance, which is the contribution of a numeric feature that differs by one standard deviation. This is a simplification of the SMOTE paper, which set the penalty to the median standard deviation of the numeric features of the class [7]. A new record is placed at a random point between a record of the class and one of its five nearest neighbours of the same class. Its numeric features are interpolated in their original units, and each nominal feature and flag takes the level held by most of the five neighbours. Every class below the target, of 5,000 or 20,000 records, is raised to it.

Tomek links are found in the same distance. Both records of every link are removed, as in [2]. SMOTE with Tomek links removes the links after SMOTE has added the same synthetic records, so the two treatments differ only by the removed links. Both are applied to the training fold only, and the validation fold is left as it is.

The removal is large. In the first fold at the target of 5,000, SMOTE added 14,648 records to the 206,138 training records, and the removal of Tomek links then took out 41,312. These included 10,801 of the 13,083 DoS records and 14,529 of the 35,620 Exploits records.

E. Verification

Each part of the implementation was checked before the experiments were run. A forest of one tree with no bootstrap and every feature gives the same predictions as the tree, and a forest gives the same predictions whether it is built on one thread or on four. The Tomek links found on small sets with ties and duplicates equal those found by applying the definition to every pair of records. The saved predictions of the k-nearest neighbours classifier reproduce its assignment 2 fold scores to four decimals on the saved folds, so the three models are compared on the same records. These checks form 24 unit tests in `src/tests`.

IV. Empirical Procedure

A. Research questions and hypotheses

The hypotheses below were written before any sweep was run, and are reported as they were written. Two open questions were added later, after a check of the implementation on the first fold, but still before the sweeps.

The first question is how many trees the forest needs. The score was expected to rise steeply over the first trees, then flatten and never decline, and the spread of the score across seeds was expected to shrink as trees were added. The question is answered by the curve of the score against the number of trees, and by the standard deviation of the score across seeds at each number of trees.

The second question is how much each source of randomness contributes. An intermediate number of features per split was expected to score best. Bagging, with all 42 features at every split, was expected to score worse, because every tree would split first on the same three features that carried about two thirds of the importance of the assignment 2 tree. Whether the bootstrap still helps was left as an open question. Since 46.5% of the records are duplicates, a bootstrap may add little diversity, while it still costs each tree about 37% of the distinct records. The evidence is the score at each number of features per split, with and without the bootstrap, so that each source of randomness is compared with the other held fixed.

The third question is whether the trees of the forest need the depth and leaf size limits of the single tree. The forest was expected to tolerate deeper trees, because averaging removes much of the variance that depth adds. The training and validation scores of the forest and the tree are compared at each depth and leaf size.

The fourth question is which treatment of the class imbalance serves the rare classes best. Class weighting and the balanced bootstrap were expected to raise the recall of the rare classes and lower their precision. The removal of Tomek links was expected to hurt Backdoor, DoS and Analysis, because most of their records sit in conflicting duplicate groups, which are removed as links at distance zero, as described in Section II-F. The treatments are compared on the macro-averaged F_1 score, on the recall, precision and F_1 score of the four rarest classes, and, for Tomek links, on the scores of the three named classes against SMOTE alone.

The fifth question, also left open, is whether selecting the forest on accuracy instead of on the macro-averaged F_1 score leads to a different forest. Accuracy rewards the common classes, so the two measures may prefer different settings. The selection rule is applied to every sweep once with each measure, and the per-class scores are examined wherever the two selections differ.

B. Cross-validation protocol

All three models are scored on the five stratified folds of assignment 2, which were drawn once with a fixed seed, saved, and read by every experiment. Each record therefore falls in the same fold for every model, and the scores of two models on a fold form a pair, as the paired test of Section II-H requires.

The forest is random, so a single fit would mix the effect of a setting with the luck of one seed. Every configuration of the forest was therefore fitted with five seeds on each of the five folds, which is 25 fits. The five scores of a fold were first averaged over the seeds, and the mean and the standard error of a configuration were then taken over the five fold averages. The tree uses every feature at every split and no bootstrap, so it was fitted once per fold.

The same five folds serve both to tune the models and to evaluate the selected ones. The selected configuration is the one that scored best on these folds, so its score on the same folds is biased upwards. The tree was tuned on the same folds under the same rule, so the forest and the tree carry the same kind of bias.

C. Control parameters and sweeps

A full grid over the six control parameters of the forest would take weeks of computing, so the parameters were swept one research question at a time, in the order of the questions. Each sweep holds every other parameter at the value selected by the sweeps before it, or at the usual default where no sweep has yet run. Table I lists the values tried. The forest uses the entropy as its impurity throughout.

The tree was tuned again for this report, with a full grid of 240 configurations over its four parameters, because it is both a baseline and the building block of the forest.

The depth and leaf size sweeps of the forest use the values of the tree grid, so the two models are compared at every value. The balanced bootstrap sizes start at 139 records per class, the size of the smallest class in a training fold, and rise towards sizes at which the balanced bootstrap approaches class weighting. The SMOTE target of 5,000 raises the four smallest classes, and the target of 20,000 raises seven.

Sweeping one parameter at a time has a cost. The number of trees was selected at six features per split with the bootstrap, before either was tuned, and was not revisited. The final evaluation therefore records the score after every number of trees up to 500, to check that the score has still levelled off at the selected configuration.

TABLE I: Control parameter values searched in each sweep

Sweep	Parameter	Values
0	Tree grid	Depth 5, 8, 10, 12, 15, 20, 25, 30, 40, none; leaf size 1, 2, 5, 10, 20, 50; Gini, entropy; class weights none, balanced
1	Trees	1, 2, 5, 10, 20, 50, 100, 200, 300, 400, 500
2	Features per split	1, 3, 6, 10, 14, 21, 42, each with and without the bootstrap
3a	Maximum depth	5, 8, 10, 12, 15, 20, 25, 30, 40, none
3b	Minimum leaf size	1, 2, 5, 10, 20, 50
4	Imbalance treatment	None; class weights; balanced bootstrap at 139, 500, 1,000, 2,500 and 5,000 per class; SMOTE to 5,000 and 20,000; SMOTE with Tomek links at both targets

D. Selection rule

Every sweep is resolved by the one-standard-error rule of Section II-H, applied to the mean validation macro-averaged F_1 score. The configuration with the highest mean is found first, and the simplest configuration whose mean lies within one standard error of it is selected. Simplest means fewer trees, fewer features per split, a smaller maximum depth, a larger minimum leaf size, or no imbalance treatment. The bootstrap is the one exception, because neither setting is simpler than the other. It is kept as the default, since the questions were posed for the forest of [4], and it is dropped only if dropping it scores higher by more than one standard error.

The tree grid varies four parameters at once, so no single parameter orders it. Simplest there means the configuration with the fewest leaves, averaged over the five folds, among those within one standard error of the best.

A selected value that is the first or the last of its sweep may mean that the best value lies outside the range tried, and in that case the sweep was to be extended in that direction. No extension was needed. The one selection at the end of its range was a minimum leaf size of one, which is the smallest possible value.

Only the selections made on the macro-averaged F_1 score are carried forward to the next sweep.

E. Final evaluation and comparison

The selected forest and the tuned tree were then fitted again on each of the five training folds and scored on the matching validation folds. The k-nearest neighbours classifier enters through its saved assignment 2 predictions on the same folds.

The forest is compared with each of the two earlier models by a paired t-test on the five fold scores of the macro-averaged F_1 score. The two p-values are adjusted by the Holm correction, and a difference is taken as significant at the 5% level.

Three further analyses were planned before the final runs. The first relates the gain of the forest over the tree on each class to the share of the records of that class in conflicting duplicate groups, using the Spearman rank correlation. The second measures how widely the importance is spread, by the share of the three most important features and by the number of features needed to reach 90% of the importance.

The third measures the time to fit each model and to classify the validation records of the first fold. The tree and the forest were timed on all eight cores of the same machine, while the k-nearest neighbours classifier was timed on one thread in assignment 2, so its prediction time is not directly comparable.

V. Results and Discussion

A. Number of trees

The score of the forest rises steeply over the first trees and then levels off. In the first sweep, with six features per split and the bootstrap, the macro-averaged F_1 score rises from 0.5256 for one tree to 0.5769 at 50 trees, and stays between 0.5752 and 0.5767 from 100 to 500 trees, as shown in Figure 1. As Section IV-A anticipated, the curve flattens and does not decline beyond the noise. The one-standard-error rule selects 20 trees, which score 0.5736. The spread of the score across seeds broadly shrinks, from 0.0040 at one tree to 0.0009 at 500.

The plateau follows from the variance of an average of correlated trees, as described in Section II-D. The part of the variance that falls with the number of trees is already small after a few tens of trees. What remains is the part set by the correlation between the trees, which adding trees cannot lower.

Two trees score below one, at 0.5129 against 0.5256. With two trees, every record on which the trees disagree is a tied vote, and the tie goes to the class with the lowest code, as described in Section III-C. Normal and Reconnaissance hold the two lowest codes, so whenever one of the two trees predicts either class, the tie goes to that class, whether or not it is right. A third tree breaks most such ties, and at five trees the forest scores 0.5584, above a single tree.

The number of trees was chosen before the features per split and the bootstrap were tuned, so the curve was measured again for the final configuration, with 14 features per split and no bootstrap. The forest scores 0.6102 at 20 trees, 0.6120 at 50, 0.6141 at 100 and 0.6147 at 500. The final curve therefore levels off later, at about 100 trees. At 20 trees the forest lies 0.0045 below its score at 500 trees, which is 0.0006 more than the standard error of 0.0039. The forest was kept at 20 trees, as selected by the procedure fixed in advance.

B. Sources of randomness

Figure 2 gives the score of the forest against the number of features per split, with and without the bootstrap. The score rises with the number of features up to 14 or 21 and falls at 42, so an intermediate value is best, as the second hypothesis stated. The best value lies more than twice the usual default, however. Without the bootstrap the forest scores 0.5841 at six features, 0.6125 at 14 and 0.6159 at 21, and the one-standard-error rule selects 14. At the default of six features with the bootstrap, the forest scores 0.5736, below the 0.5860 of the single tuned tree. The usual setting of a random forest is therefore worse than one tree on this data.

A possible cause is that the signal is concentrated in a few features. The service of a flow, the source byte count and the source time to live carried about two thirds of the importance

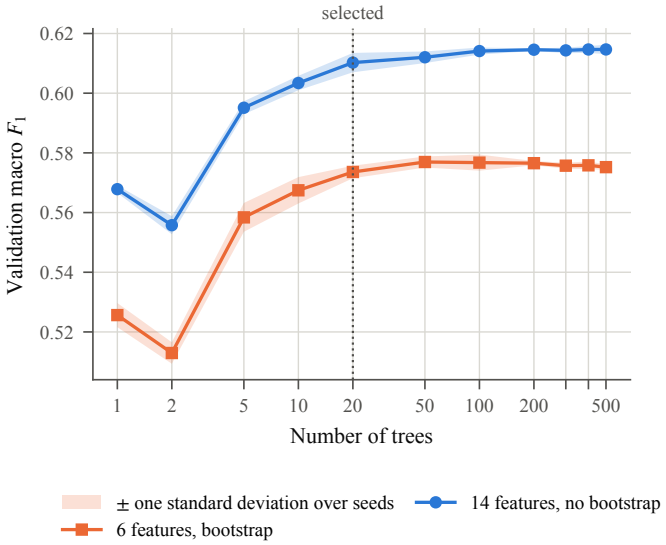


Fig. 1: Validation macro-averaged F_1 score against the number of trees

of the assignment 2 tree. When six of the 42 features are drawn at a split, none of these three is among them at about 62% of the splits, against about 29% when 14 are drawn. Each tree is then weaker, and the lower correlation between the trees does not make up for it. Section II-E describes the trade-off between the strength of the trees and their correlation.

Section IV-A anticipated that bagging, with all 42 features at every split, would score below an intermediate number of features. Instead, bagging scores 0.6085, level with the best forest with the bootstrap, at 0.6090 with 21 features. The three shared features therefore did not lower the score of bagging by a measurable amount.

Dropping the bootstrap raised the score at every number of features from 1 to 21, by between 0.0068 and 0.0105. Only at 42 features did the bootstrap help, by 0.0244. That forest scores 0.5841, level with a single tree grown without limits, as Section II-E predicts.

The bootstrap may cost each tree more than it adds. Each tree misses about 37% of the distinct records, which makes it weaker. The diversity it adds is smaller than usual, because 46.5% of the records are duplicates, and a record that the sample missed often has an identical copy that was drawn. Once the features are subsampled, the trees are already diverse enough without it.

C. Tree complexity

The tree grid selects the entropy, a maximum depth of 25 and a minimum leaf size of 5, without class weights. This tree scores 0.5860, within one standard error of the best tree at 0.5883. The best trees with the Gini index and with class weights also lie within one standard error, in line with the small differences between the two impurity measures reported in [22].

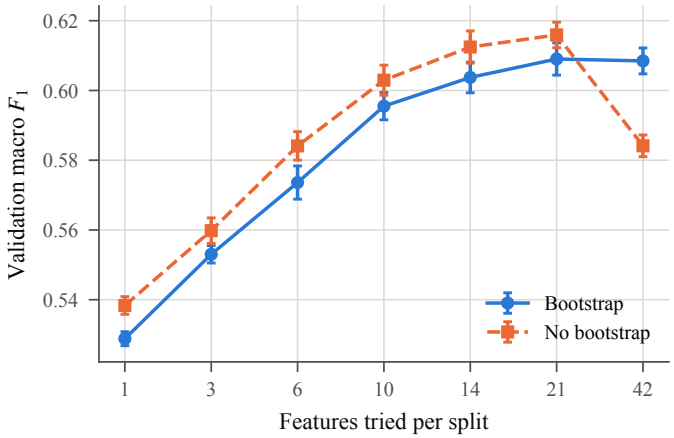


Fig. 2: Validation macro-averaged F_1 score against the number of features per split

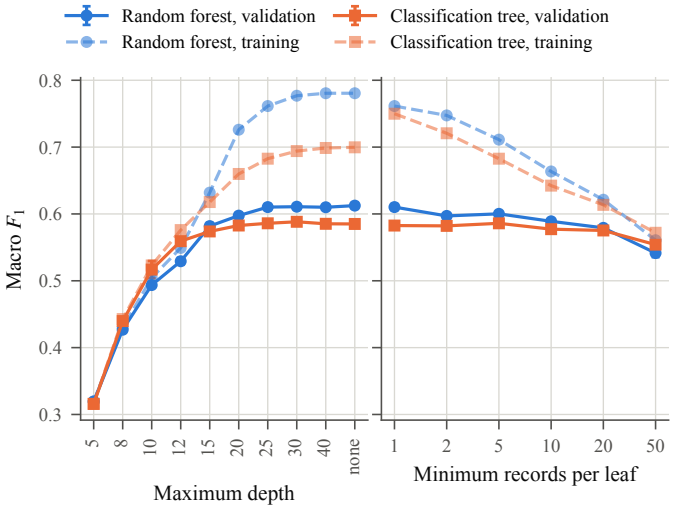


Fig. 3: Training and validation macro-averaged F_1 score against maximum depth and minimum leaf size

The left panel of Figure 3 shows that depth beyond 25 neither helps nor harms either model. From depth 25 to unlimited depth, the tree scores between 0.5850 and 0.5883 and the forest between 0.6100 and 0.6125, and both are selected at depth 25. Section IV-A anticipated that the forest would tolerate deeper trees than the single tree. Instead, the single tree is not hurt by depth either, so the two models select the same depth.

The leaf size separates the two models, as the right panel of Figure 3 shows. The score of the forest falls from 0.6102 with one record per leaf to 0.5970 with two and 0.5415 with 50, and the rule selects one record per leaf. For the tree the leaf size matters less, since at depth 25 it scores 0.5826 with one record per leaf and 0.5860 with five. A likely explanation is the weight that one leaf carries. A leaf of one record follows the noise in the training records closely. In a single tree such a leaf decides the class of every record that reaches it, while in the forest it is one vote among twenty.

The forest also fits its training records more closely than the tree. Its training score is 0.7614 against 0.6827 for the tree, which leaves a difference of 0.1512 from its validation score, against 0.0967 for the tree. The forest has both the larger difference and the higher validation score, because averaging removes part of the variance of its fully grown trees without removing their fit to the training records.

D. Imbalance treatment

No treatment of the imbalance raised the macro-averaged F_1 score above the 0.6102 of the forest without one, as shown in Table II. SMOTE to 5,000 records per class comes closest, at 0.6051, which is within one standard error. SMOTE to 20,000 follows at 0.5985, and class weighting at 0.5880. The balanced bootstrap scores lowest, from 0.4825 with 139 records per class to 0.5735 with 5,000. SMOTE with Tomek links scores 0.5711 and 0.5613 at the two targets. The rule therefore selects no treatment. The balanced bootstrap and SMOTE each had several settings to choose from, against one for class weighting, which favoured them, and they still fell short.

The rare classes below are the four smallest, Worms, Shellcode, Backdoor and Analysis, and their scores are the unweighted means over the four. Class weighting and the balanced bootstrap raise the recall of the rare classes from 0.36 without treatment to between 0.43 and 0.65, and lower their precision from 0.69 to between 0.11 and 0.35, which is the pattern the fourth hypothesis described. Precision falls faster than recall rises, so the F_1 score of the rare classes falls, from 0.41 to between 0.18 and 0.36. The treatments move records of the common classes into the rare classes, and the common classes are large enough that a small share of them outnumbers the rare records gained.

The balanced bootstrap improves as the number of records per class rises. With 139 records per class, each tree is grown on 1,390 records and ends with about 466 leaves, against about 11,700 without a treatment, so each tree sees too few records to learn the common classes.

As the fourth hypothesis stated, the removal of Tomek links lowers the score of Backdoor, DoS and Analysis. Table III compares SMOTE alone with SMOTE and Tomek links at 5,000 records per class. Against SMOTE alone, the F_1 score falls from 0.316 to 0.169 for DoS, from 0.157 to 0.101 for Backdoor, and from 0.195 to 0.102 for Analysis. The removal took out 41,312 records in the first fold, as reported in Section III-D. On the cleaned training records the forest scores 0.9792, while its validation score falls. The removal clears the conflicting groups from the training records, but the validation records still hold them, as described in Section II-F. The forest therefore learns a problem that is easier than the one it is tested on.

The removal also changes how the forest divides records between DoS and Backdoor. The recall of Backdoor rises from 0.108 under SMOTE alone to 0.614, while its precision falls to

TABLE II: Macro-averaged F_1 score of each imbalance treatment, with mean scores over the four rarest classes

		Rare classes		
Treatment	Macro F_1	Recall	Precision	F_1
None	0.6102	0.359	0.693	0.411
Class weights	0.5880	0.426	0.351	0.363
Balanced bootstrap, 139	0.4825	0.645	0.107	0.176
Balanced bootstrap, 500	0.5140	0.617	0.145	0.231
Balanced bootstrap, 1,000	0.5342	0.595	0.179	0.272
Balanced bootstrap, 2,500	0.5579	0.562	0.225	0.316
Balanced bootstrap, 5,000	0.5735	0.538	0.263	0.345
SMOTE, 5,000	0.6051	0.405	0.489	0.399
SMOTE, 20,000	0.5985	0.412	0.325	0.358
SMOTE + Tomek, 5,000	0.5711	0.578	0.301	0.357
SMOTE + Tomek, 20,000	0.5613	0.538	0.268	0.335

TABLE III: Per-class scores under SMOTE and SMOTE with Tomek links, at 5,000 records per class

Class	Treatment	Precision	Recall	F_1
Backdoor	SMOTE	0.287	0.108	0.157
	+ Tomek	0.055	0.614	0.101
DoS	SMOTE	0.369	0.276	0.316
	+ Tomek	0.723	0.095	0.169
Analysis	SMOTE	0.542	0.119	0.195
	+ Tomek	0.063	0.287	0.102

0.055, and the recall of DoS falls from 0.276 to 0.095. The way the removal empties the conflicting groups may explain this outcome. It leaves no real records where the groups lay, while the synthetic Backdoor records added by SMOTE, which have no identical copies, remain nearby. The forest then assigns those regions, and the many DoS records in them, to Backdoor.

E. Choice of performance measure

Selecting on accuracy instead of on the macro-averaged F_1 score changes the selection in three of the six sweeps, as shown in Table IV. The two measures agree on the features per split and the bootstrap, on the leaf size of the forest and on the imbalance treatment. They disagree on the other three. For the tree grid, accuracy selects depth 15 with one record per leaf, where the macro-averaged F_1 score selects depth 25 with five. Accuracy also selects 100 trees rather than 20, and a forest depth of 20 rather than 25.

Where the measures disagree on depth, accuracy prefers the shallower trees, and the per-class scores in Table V show which classes lose by the choice. In the forest, the shallower trees lower the F_1 score of Analysis from 0.204 to 0.115 and that of DoS from 0.329 to 0.297. Only Exploits gains, from 0.722 to 0.732. The tree loses on the same two classes, with Analysis falling from 0.170 to 0.088 and DoS from 0.318 to 0.277, while Exploits rises from 0.707 to 0.731. Accuracy rises by at most 0.0049, because Exploits holds 17% of the records and Analysis and DoS together about 7%. A possible cause is that Analysis and DoS are separated from Exploits only by the last few splits of a deep tree. A shallower tree leaves those regions to Exploits, the larger class. Accuracy counts records,

TABLE IV: Configurations selected by each performance measure

Sweep	Parameter	Macro F_1	Accuracy
0	Tree grid	Depth 25, leaf 5	Depth 15, leaf 1
1	Trees	20	100
2	Features, bootstrap	14, none	14, none
3a	Maximum depth	25	20
3b	Minimum leaf size	1	1
4	Treatment	None	None

TABLE V: F_1 score of three classes under the selection made by each measure

	Tree		Forest	
Class	Macro F_1	Accuracy	Macro F_1	Accuracy
Analysis	0.170	0.088	0.204	0.115
DoS	0.318	0.277	0.329	0.297
Exploits	0.707	0.731	0.722	0.732

so it rewards the shallower trees, while the macro-averaged F_1 score counts classes, so it does not.

The disagreement on the number of trees has a different cause. The standard error of accuracy over the folds is about 0.001, against about 0.004 for the macro-averaged F_1 score, so the one-standard-error rule leaves accuracy a narrower margin around its best value, at 300 trees. Accuracy selects 100 trees, while the macro-averaged F_1 score accepts 20. The choice of measure therefore decides the depth of both models, and which of the conflicting classes they serve.

F. Comparison with the assignment 2 models

1) Overall performance: The forest reaches the highest score of the three models on every performance measure. Table VI reports a macro-averaged F_1 score of 0.6102 for the forest, against 0.5860 for the tree and 0.5010 for the k-nearest neighbours classifier. The same order holds for accuracy, where the forest reaches 0.8315 against 0.8166 for the tree and 0.7717 for the k-nearest neighbours classifier.

The gain over both models is larger than the variation between folds. The forest scores higher than the tree and the k-nearest neighbours classifier on each of the five folds, as shown in Table VII. Table VIII reports a paired t-test, which puts the mean difference at 0.0243 between the forest and the tree, and at 0.1093 between the forest and the k-nearest neighbours classifier, with Holm-adjusted p-values [16] of 0.00015 and 0.00005. The forest is therefore the strongest of the three models on this data.

The size of the gain is more visible when considering the highest score that a model can achieve. The conflicting duplicate groups limit the maximum achievable accuracy to 0.9139, as described in Section III-A. The accuracy of the tree is 0.0973 below this bound and that of the forest 0.0824 below it, so the forest reduces the difference from the bound by about 15%. Nor is the gain the result of a favourable seed: across the five seeds, the macro-averaged F_1 score of the forest has a standard deviation of 0.0032 against a gain of 0.0243 over the

TABLE VI: Mean performance over the five folds, with the standard deviation in brackets

Measure	k-NN	Tree	Forest
Macro F_1	0.5010 (0.0029)	0.5860 (0.0073)	0.6102 (0.0101)
Weighted F_1	0.7747 (0.0020)	0.8091 (0.0014)	0.8233 (0.0020)
Accuracy	0.7717 (0.0026)	0.8166 (0.0009)	0.8315 (0.0021)
Balanced accuracy	0.5097 (0.0030)	0.5647 (0.0042)	0.5837 (0.0075)

The forest scores are means over five seeds. Across the seeds, its macro-averaged F_1 score has a standard deviation of 0.0032, from 0.6069 to 0.6139.

TABLE VII: Macro-averaged F_1 score per fold

Fold	k-NN	Tree	Forest
1	0.5037	0.5854	0.6076
2	0.5043	0.5880	0.6118
3	0.4988	0.5965	0.6234
4	0.5002	0.5762	0.5954
5	0.4980	0.5838	0.6130

tree, as shown in Table VI. The forest therefore improves on the tree by a margin that holds on every fold, but it remains 0.0824 below the best attainable accuracy.

2) Performance per class: The forest scores higher than the tree in every one of the ten classes, as shown in Table IX. Although a class with many conflicting records leaves less room for any model to improve, the gains do not follow the conflict rate. The largest gains are in Shellcode (0.069), Fuzzers (0.042), Analysis (0.035) and Worms (0.030), classes with conflict rates from 0.9% to 77.0%, and the rank correlation between the gain and the conflict rate is close to zero (Spearman $\rho = -0.02$, $p = 0.96$).

The three classes with the most conflicting records remain the weakest for every model. Analysis, DoS and Backdoor have conflict rates of 77.0%, 77.4% and 84.9%, and the forest reaches F_1 scores of only 0.204, 0.329 and 0.177 on them. Table IX also shows where their records go, since the forest predicts 51% of Analysis, 66% of DoS and 56% of Backdoor records as Exploits. Records with identical features cannot be separated by any classifier, which makes these errors a property of the data rather than of the model. The forest also predicts Analysis rarely but accurately, with a precision of 0.896 against a recall of 0.116.

DoS is the one class on which the k-nearest neighbours classifier scores higher than both tree models, with an F_1 score of 0.371 against 0.329 for the forest and 0.318 for the tree.

3) Feature importance: The forest spreads its importance over more features than the tree, as shown in Table X. The three leading features, the service of a flow, the source byte count and the source time to live, account for 0.68 of the importance of the tree but 0.38 of that of the forest. The tree needs 10 features to reach 0.90 of its importance, and the forest needs 22. The service of a flow falls from 0.30 of the importance in the tree to 0.13 in the forest, and the source time to live from 0.18 to 0.09.

The spread follows from the features drawn at each split. With 14 of the 42 features drawn, none of the three leading features is available at about 29% of the splits, as found in Section V-B. At those splits the tree must split on another feature. Features that the single tree barely used then gain importance. The number of connections with the same destination address and source port rises from 0.002 of the importance in the tree to 0.055 in the forest, and the number of connections in each state for a range of time to live values rises from 0.001 to 0.042. The second of these is built from the time to live, so it may stand in for the source time to live when that feature is not drawn.

TABLE VIII: Paired t-tests on the macro-averaged F_1 score over the five folds, with Holm-adjusted p-values

Comparison	Difference	95% interval	t	p
Forest − tree	0.0243	0.0194 to 0.0291	13.93	0.00015
Forest − k-NN	0.1093	0.0954 to 0.1231	21.92	0.00005

TABLE IX: Conflict rate, F_1 score per class, and share of each class that the forest predicts as Exploits

		F_1 score			Forest to
Class	Conflict (%)	k-NN	Tree	Forest	Exploits (%)
Generic	0.6	0.979	0.988	0.990	1
Shellcode	0.9	0.423	0.607	0.676	13
Normal	1.0	0.896	0.920	0.934	0
Worms	8.6	0.244	0.556	0.586	37
Fuzzers	14.9	0.516	0.605	0.647	9
Reconnaissance	16.1	0.672	0.827	0.838	17
Exploits	37.1	0.658	0.707	0.722	81
Analysis	77.0	0.154	0.170	0.204	51
DoS	77.4	0.371	0.318	0.329	66
Backdoor	84.9	0.096	0.161	0.177	56

The ten features removed from the k-nearest neighbours classifier in assignment 2 account for 0.10 of the importance of the tree and 0.15 of that of the forest. Their removal therefore discarded little of the information that the trees use. These shares carry the bias of impurity-based importance described in Section II-C.

4) Computational cost: The forest takes about twice as long as the tree to fit, and both classify records more than a hundred times faster than the k-nearest neighbours classifier. Table XI reports the costs on the first fold. The tree took 3.7 seconds to fit and the 20-tree forest 7.1 seconds. The forest grows twenty trees in about twice the time of one, because its trees are built in parallel on eight cores and each of its splits searches 14 features in place of 42. A forest of 500 trees, which Section V-A found to score about 0.0045 higher, took about 170 seconds to fit, so the selected forest costs about 4% of that.

The forest classified 2.6 million records per second, against 13.8 million for the tree and 2,111 for the k-nearest neighbours classifier. Even with a perfect use of eight threads in place of its one, the classifier would remain more than a hundred times slower than the forest. The difference follows from how each model classifies. A tree follows one path of at most 25 tests, and the forest follows one such path in each of its 20 trees, while the k-nearest neighbours classifier measures the distance to every one of about 206,000 training records, as described

TABLE X: Feature importance of the tree and the forest

Feature or summary	Tree	Forest
Service of a flow	0.30	0.13
Source byte count	0.19	0.16
Source time to live	0.18	0.09
Same destination and source port	0.002	0.055
Connections per state and TTL	0.001	0.042
Share of the top three	0.68	0.38
Features to reach 0.90	10	22
Share of the ten removed for k-NN	0.10	0.15

TABLE XI: Fit time and prediction rate on the first fold

Model	Fit (s)	Records per second
k-NN (one thread)	–	2,111
Tree	3.7	13.8 million
Forest, 20 trees	7.1	2.6 million
Forest, 500 trees	170	–

in Section II-B.

G. Limitations

As described in Section IV-B, the same five folds were used to tune the models and to evaluate them, so the reported scores are biased upwards and are not estimates on unseen records. The tree was selected from 240 configurations and the forest from 52, so the tree had more chances to fit the folds, and the bias is unlikely to explain the higher score of the forest.

The parameters were tuned one at a time, so their interactions were only partly searched. The number of trees was checked again for the final configuration, and lies 0.0045 below the score of 500 trees, as reported in Section V-A. The depth and the leaf size were not checked in the same way, and both were selected with 20 trees.

The paired t-tests treat the five fold differences as independent, although any two training folds share three quarters of their records, so their p-values are too small, as noted in Section II-H. The forest scores higher than both models on all five folds, so the conclusion does not rest on the p-values alone.

The k-nearest neighbours classifier was not tuned again. In assignment 2 it was tuned on a stratified subsample of 40,000 records, where one neighbour led three by 0.0002, well inside the noise. Tuning on the full training folds might change its score, but a change as large as its deficit of 0.1093 is unlikely. A class weighting of its vote, which would favour the rare classes, has no effect at one neighbour, because a single vote cannot be outweighed. It would need a larger k, and the classifier scored lower at every larger k in assignment 2.

As described in Section III-D, the SMOTE distance gives each nominal mismatch the weight of one standard deviation, a simplification of the SMOTE paper. A different weight would change which records are neighbours, and therefore both the synthetic records and the Tomek links found.

The importances are based on the decrease in impurity, which favours features with many distinct values, as described in Section II-C. The service of a flow and the source time to live have only 13 distinct values each, against up to 121,356 for other features, so the bias does not explain their high importance. The importance of the features with many values, such as the source byte count with 9,382, is likely to be overstated.

The results hold for one dataset, whose attacks were generated synthetically and whose ordinary traffic was recorded in one laboratory, as described in Section II-A. The share of duplicated records, 46.5%, drives two of the findings, the weak bootstrap and the failure of the removal of Tomek links, and may be lower in traffic recorded elsewhere.

VI. Conclusion

A random forest was tuned and evaluated on the UNSW-NB15 dataset and compared with the classification tree and the k-nearest neighbours classifier of assignment 2. The selected forest of 20 trees, with 14 features per split, no bootstrap and trees of depth 25, scores 0.6102 on the macro-averaged F_1 score, against 0.5860 for the tree and 0.5010 for the k-nearest neighbours classifier, and scores higher than both on every fold. Its score levels off after a few tens of trees. Although bagging was expected to score worse, it scored level with the best forest that uses the bootstrap, and dropping the bootstrap raised the score at every number of features below 42. The forest needed the same depth as the tree and gained instead from leaves of one record. None of the imbalance treatments improved on using no treatment, and SMOTE with Tomek links lowered the score of the three classes with the most conflicting records. Selecting on accuracy chose shallower trees, at the cost of Analysis and DoS.

The forest reduces the difference between the accuracy of the tree and the best attainable accuracy of 0.9139 by about 15%. Analysis, DoS and Backdoor, most of whose records sit in conflicting duplicate groups, remain the weakest classes for every model, and their errors are a property of the data rather than of the model.

Four steps would extend this work. The first is to evaluate the selected models on records held out from tuning, by a separate test split or by nested cross-validation. The second is to tune the number of trees together with the features per split and the bootstrap. The third is to resolve the conflicting duplicate groups and measure how much of the remaining error they explain. The fourth is to replace the majority vote by the average of the class proportions in the leaves, which breaks ties by the confidence of the trees.

References

[1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim, "On the surprising behavior of distance metrics in high dimensional space," in *Proceedings of the Eighth International Conference on Database Theory*, ser. Lecture Notes in Computer Science, vol. 1973. Springer, 2001, pp. 420–434.

[2] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD Explorations Newsletter*, vol. 6, no. 1, pp. 20–29, 2004.

[3] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[4] ——, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[5] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, California: Wadsworth International Group, 1984.
[6] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys and Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
[7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
[8] C. Chen, A. Liaw, and L. Breiman, "Using random forest to learn imbalanced data," Department of Statistics, University of California, Berkeley, Tech. Rep. 666, 2004.
[9] T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
[10] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
[11] ——, "Ensemble methods in machine learning," in *Proceedings of the First International Workshop on Multiple Classifier Systems*, ser. Lecture Notes in Computer Science, vol. 1857. Springer, 2000, pp. 1–15.
[12] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.
[13] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York: Springer, 2009.
[14] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
[15] T. K. Ho, "The random subspace method for constructing decision forests," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 8, pp. 832–844, 1998.
[16] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
[17] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, vol. 2. Morgan Kaufmann, 1995, pp. 1137–1143.
[18] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proceedings of the Military Communications and Information Systems Conference*, 2015, pp. 1–6.
[19] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003.
[20] P. Probst and A.-L. Boulesteix, "To tune or not to tune the number of trees in random forest," *Journal of Machine Learning Research*, vol. 18, no. 181, pp. 1–18, 2018.
[21] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.
[22] L. E. Raileanu and K. Stoffel, "Theoretical comparison between the gini index and information gain criteria," *Annals of Mathematics and Artificial Intelligence*, vol. 41, no. 1, pp. 77–93, 2004.
[23] C. Strobl, A.-L. Boulesteix, A. Zeileis, and T. Hothorn, "Bias in random forest variable importance measures: Illustrations, sources and a solution," *BMC Bioinformatics*, vol. 8, p. 25, 2007.
[24] I. Tomek, "Two modifications of CNN," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 6, no. 11, pp. 769–772, 1976.